

10월 29일 발표

김민규

Classical solver

angle of attack encountered in subsonic flight conditions. We then run the simulations with the help of the steady-state RANS solver *simpleFOAM* via the SIMPLEC algorithm Caretto et al. [1973], Doormaal and Raithby [1984] and with the $k - \omega$ SST turbulence model Menter et al. [2003] until convergence of drag and lift coefficients. Simulations are done on 16 CPU cores of an AMD Ryzen™ Threadripper™ 3960X. Figure 2 shows a near view of the pressure and x -component of the velocity

AIRFRANS: High Fidelity Computational Fluid Dynamics
Dataset for Approximating Reynolds-Averaged-Navier-Stokes
Solutions (36th Conference on Neural Information Processing
Systems (NeurIPS 2022))

OOD data

- *Full data regime*: 800 samples for the training set and 200 for the test set
- *Scarce data regime*: 200 samples for the training set and the same test set as in the full data regime
- *Reynolds extrapolation*: the training set is composed of the samples with a Reynolds of three to five million, and the test set is formed by the samples with a Reynolds of two to three and five to six million
- *Angle of attack extrapolation*: the training set is composed by the samples with an angle of attack between -2.5° and 12.5° and the test set is composed by the samples with an angle of attack from -5° to -2.5° and 12.5° to 15° .

Self-attention

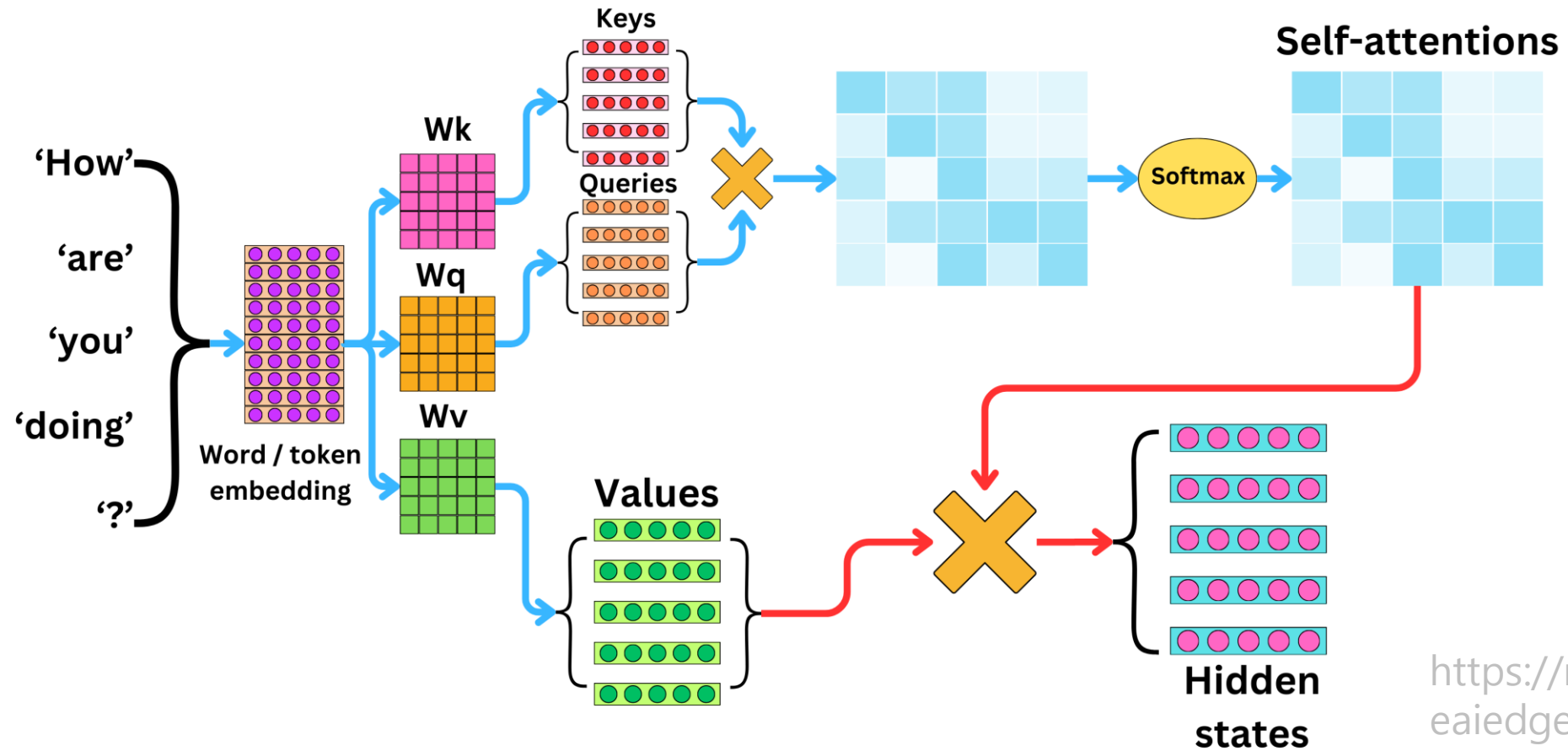
- 연속된 데이터 안에서 각 단어(토큰)가 다른 단어들과의 관계를 스스로 학습하는 메커니즘
- 문맥을 파악할 수 있음

Ex)

“인도로 걷다”

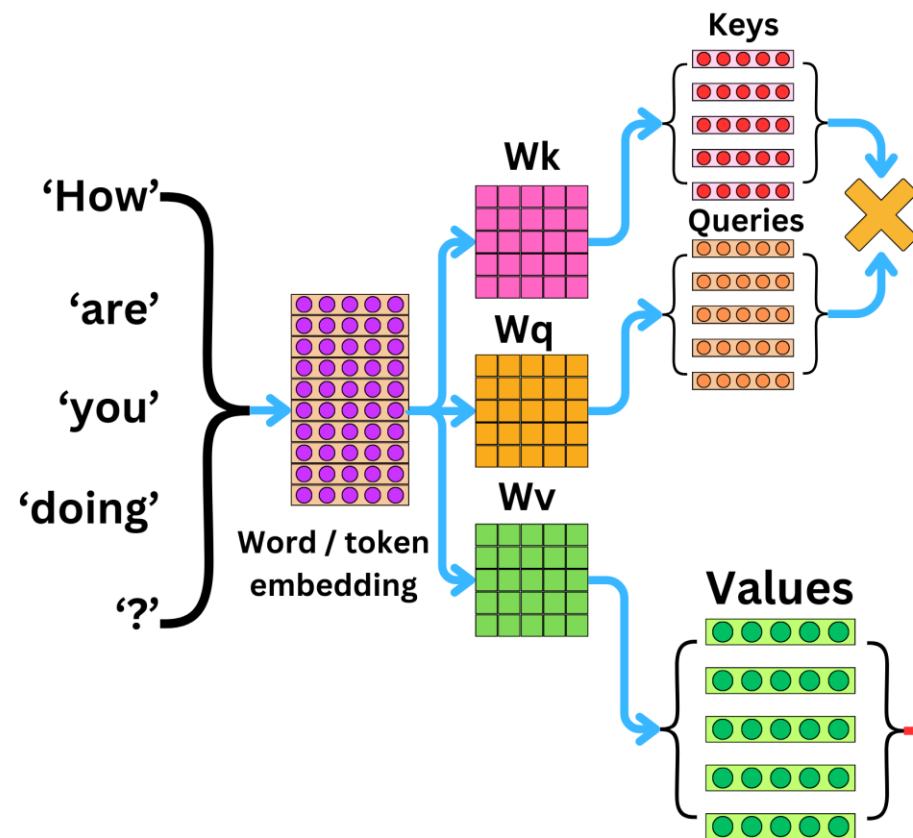
“인도 여행 ”

Self-attention



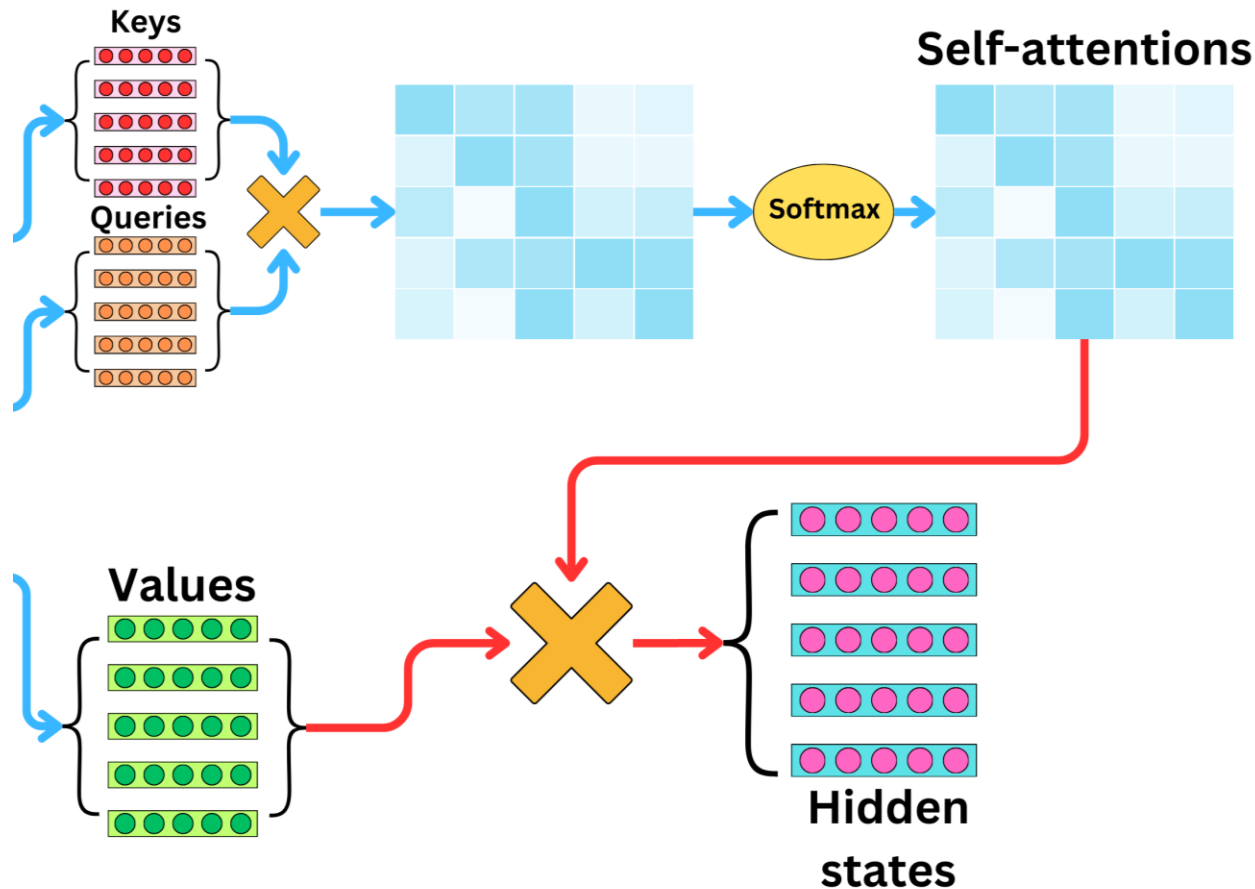
<https://newsletter.theaiedge.io/p/understanding-the-self-attention>

Self-attention



1. 각 단어/토큰을 임베딩 벡터로 변환. 각 단어는 각각의 임베딩 벡터가 존재. 비슷한 의미를 가진 벡터는 비슷한 방향을 가지도록 학습되어 있음.
2. 임베딩 벡터를 학습 가능한 가중치인 W_k, W_q, W_v 로 선형 변환.

Self-attention



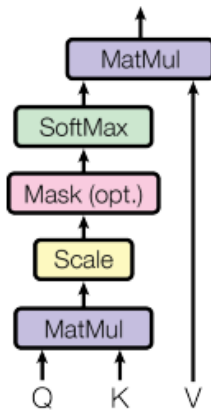
3. 선형 변환한 query 벡터와 key 벡터를 행렬 곱하고 softmax를 취해주어 확률 분포로 만들어줌. 이때 두 임베딩 벡터가 관련이 있을 경우 값이 큼.

4. 3번의 확률 분포를 value 벡터에 가중합을 하여 output를 구함.

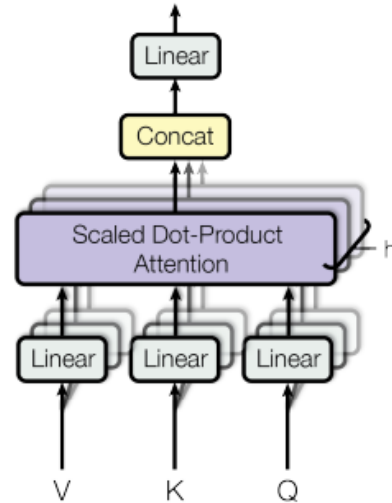
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Self-attention

Scaled Dot-Product Attention



Multi-Head Attention



$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

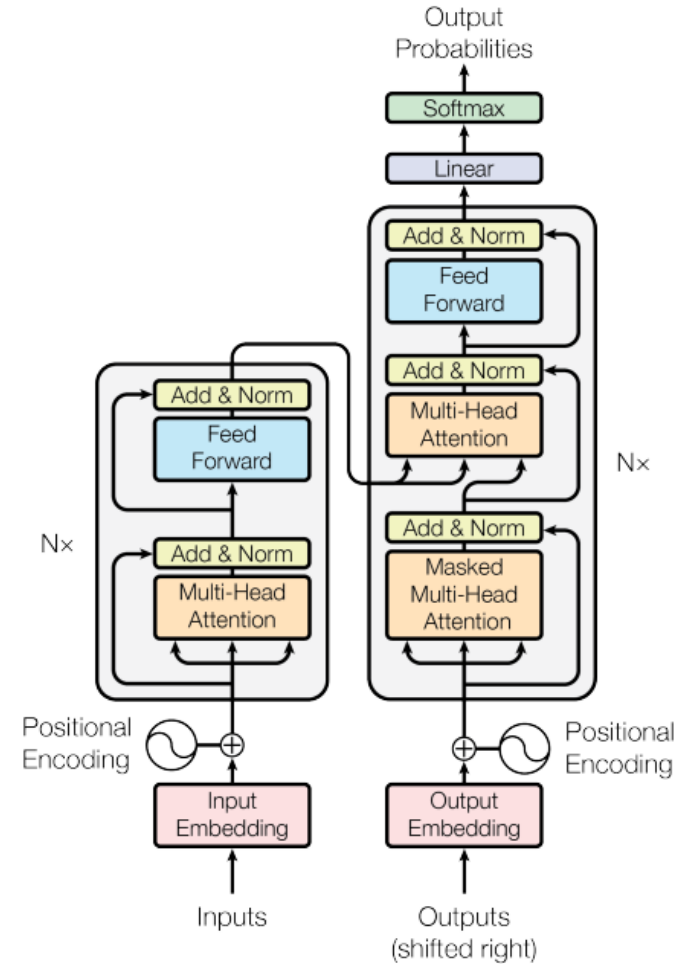
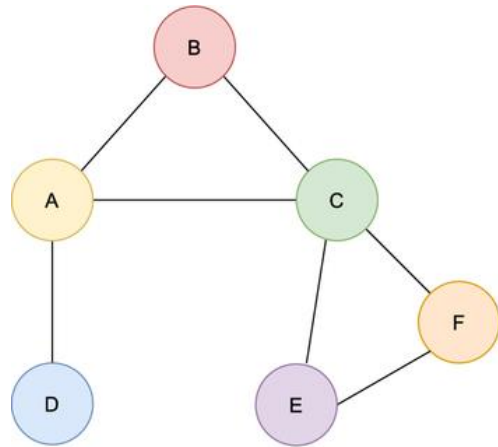
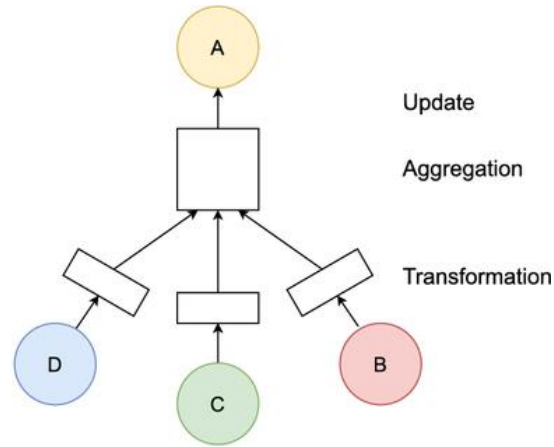


Figure 1: The Transformer - model architecture.

Graph Neural Network



Input graph



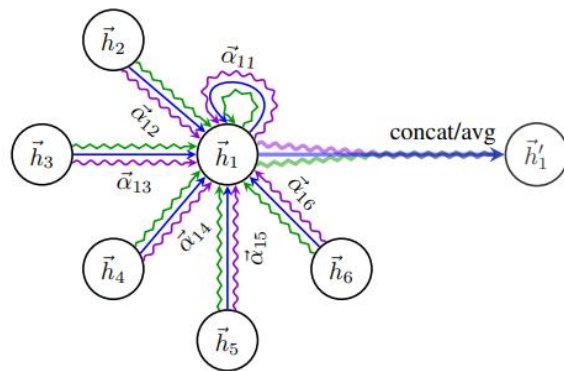
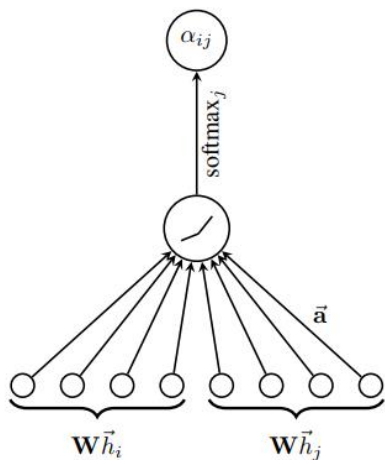
1-layer GNN

$$h_v^{(k)} = \text{Relu}(W \sum_{u \in N(v) \cup \{v\}} h_u^{(k-1)})$$

- 각각의 노드는 hidden embedding vector가 존재.
- 이웃 노드들을 aggregate하고 combine하여 update.

<https://www.researchgate.net/publication/372486484/figure/fig2/AS:11431281201950942@1698671714642/GNN-basic-structure-with-message-transformation-aggregation-and-update-52.png>

Graph attention network



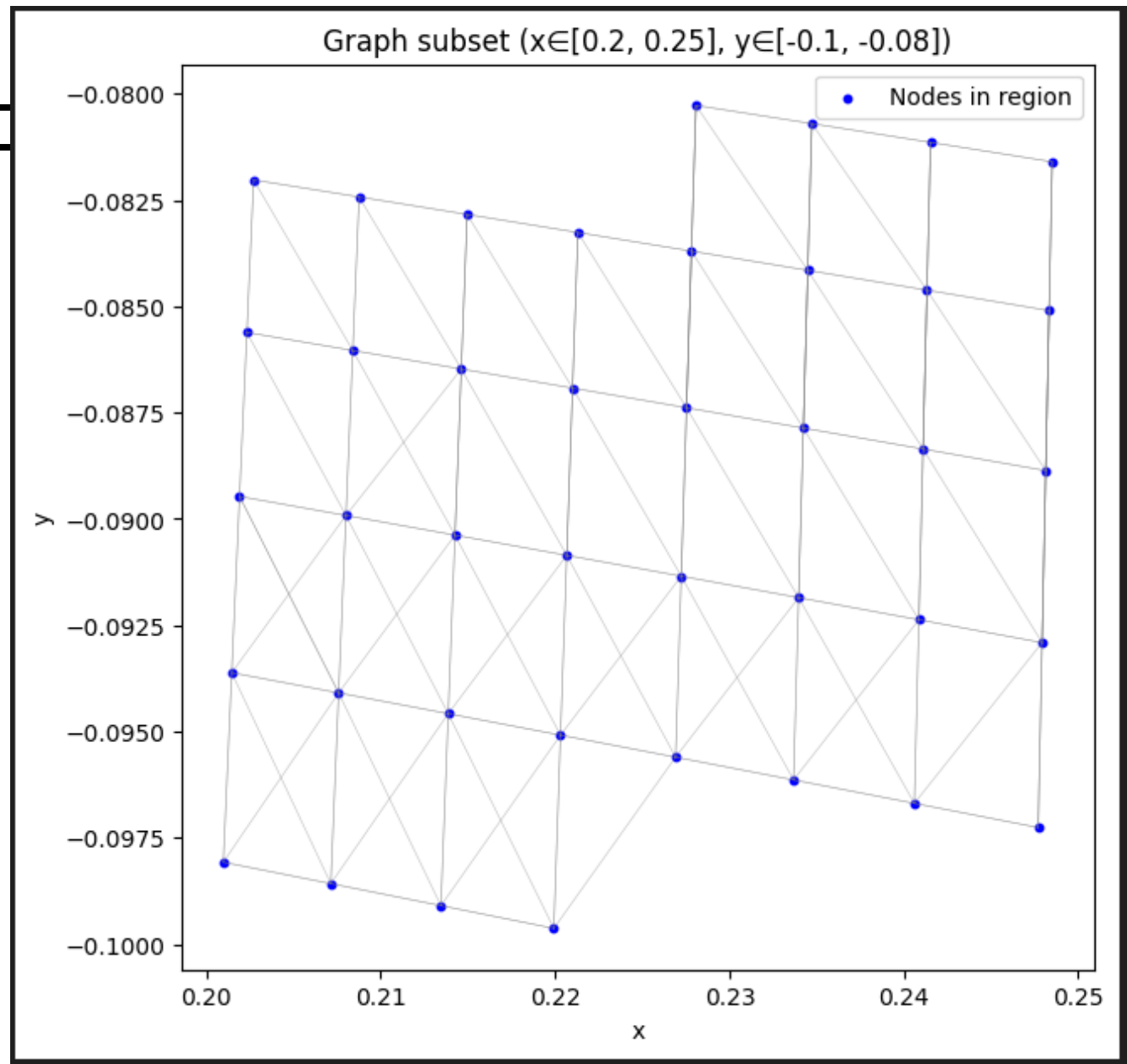
$$e_{ij} = a(W\vec{h}_i, W\vec{h}_j)$$

1. 각 hidden vector를 학습 가능한 W 로 선형 변환한 후 attention coefficient e_{ij} 를 계산

$$\alpha_{ij} = \text{softmax}_j(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i} \exp(e_{ik})}.$$

2. softmax를 취한 뒤 각각의 노드에 가중합을 취하여 hidden vector를 update.

Graph



Dataset 코드 수정

```
def _build_knn_graph(self, pos, k=10):  
    #(N, 2) numpy array를 edge_index tensor (2, E)로 반환  
    from sklearn.neighbors import NearestNeighbors  
    nbrs = NearestNeighbors(n_neighbors=k + 1).fit(pos)  
    distances, indices = nbrs.kneighbors(pos)  
  
    # i → j edges  
    src, dst = [], []  
    for i in range(len(indices)):  
        for j in indices[i][1:]: # skip self  
            src.append(i)  
            dst.append(j)  
    edge_index = torch.tensor([src, dst], dtype=torch.long)  
    return edge_index
```



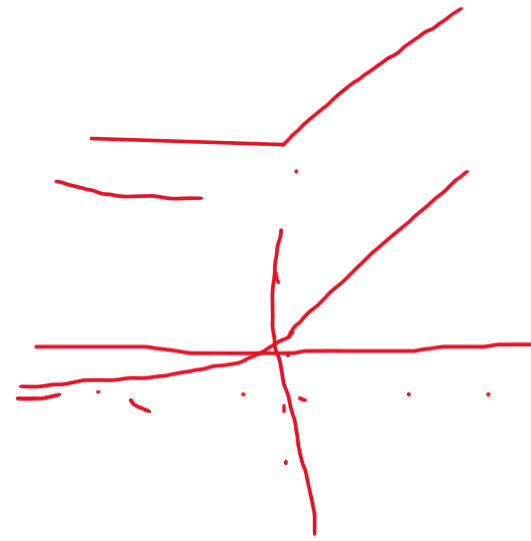
```
import torch_geometric.transforms as T
```

```
transform = T.KNNGraph(k)
```

```
graph = Data(x=x, y=y, pos=torch.tensor(simulation.position, dtype=torch.float32))  
graph = transform(graph)  
self.graphs.append(graph)
```

Gat 학습 구현

```
in_channels = 7
out_channels = 4
hidden_channels = 16
num_layers = 2
num_heads = 2
dropout = 0.1
lr = 3e-4
weight_decay = 1e-6
epochs = 20
batch_size = 4
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```



하이퍼 파라미터 설정

```
class SimpleGAT(nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels, heads=2, dropout=0.1):
        super().__init__()
        self.conv1 = GATConv(in_channels, hidden_channels, heads=heads, concat=True, dropout=dropout)
        self.conv2 = GATConv(hidden_channels * heads, out_channels, heads=1, concat=False, dropout=dropout)
        self.act = nn.ELU()
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, edge_index):
        x = self.conv1(x, edge_index)
        x = self.act(x)
        x = self.dropout(x)
        x = self.conv2(x, edge_index)
        return x
```

활성화, 드롭 아웃을 포함하는 레이어 생성

Gat 학습 구현

```
if __name__ == "__main__":
```

```
    model = run_training(train_dataset, val_dataset)
```

```
def run_training(train_dataset, val_dataset, save_path="gat.pt"):
    train_loader, val_loader = create_dataloaders(train_dataset, val_dataset, batch_size)

    model = SimpleGAT(
        in_channels=in_channels,
        hidden_channels=hidden_channels,
        out_channels=out_channels,
        heads=num_heads,
        dropout=dropout,
    ).to(device)

    optimizer = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
    criterion = nn.MSELoss()

    best_val = math.inf

    for epoch in range(1, epochs + 1):
        train_loss = train_one_epoch(model, train_loader, optimizer, criterion, device)
        val_mse = evaluate(model, val_loader, criterion, device)

        print(f"[{epoch:d}] Train {train_loss:.5f}, Val MSE {val_mse:.5f}")

        if val_mse < best_val:
            best_val = val_mse
            torch.save(model.state_dict(), save_path)

    print("학습 완료", save_path, "에 저장")
    model.load_state_dict(torch.load(save_path))
    return model
```

```
def create_dataloaders(train_dataset, val_dataset, batch_size=4):
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
    return train_loader, val_loader
```

```
def train_one_epoch(model, loader, optimizer, criterion, device):
    model.train()
    total_loss = 0
    for batch in loader:
        batch = batch.to(device)
        optimizer.zero_grad()
        out = model(batch.x, batch.edge_index)
        loss = criterion(out, batch.y)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    return total_loss / len(loader)
```

```
@torch.no_grad()
def evaluate(model, loader, criterion, device):
    model.eval()
    mse_list = []
    for batch in loader:
        batch = batch.to(device)
        out = model(batch.x, batch.edge_index)
        mse = criterion(out, batch.y).item()
        mse_list.append(mse)
    return sum(mse_list) / len(mse_list)
```

모델 생성한 후 loss가 가장 적은 모델을 저장

문제점

```
✓ 20m 46.3s
[001] Train 2222878.84998 | Val MSE 2043557.96063
[002] Train 2214182.92344 | Val MSE 2039325.70188
[003] Train 2211258.93613 | Val MSE 2034811.57906
[004] Train 2202996.88437 | Val MSE 2029596.59469
[005] Train 2200918.34520 | Val MSE 2023762.09406
[006] Train 2199789.46008 | Val MSE 2017717.84719
[007] Train 2192644.05111 | Val MSE 2011571.78125
[008] Train 2184323.91348 | Val MSE 2005335.55594
[009] Train 2174919.52102 | Val MSE 1999103.03625
[010] Train 2167729.12875 | Val MSE 1992913.80344
[011] Train 2162483.90875 | Val MSE 1986809.39969
[012] Train 2152445.73650 | Val MSE 1980757.74750
[013] Train 2153630.62771 | Val MSE 1975124.55375
[014] Train 2152566.58937 | Val MSE 1969782.12656
[015] Train 2147243.48971 | Val MSE 1964470.01594
[016] Train 2148005.65344 | Val MSE 1959223.85594
[017] Train 2129996.32123 | Val MSE 1954193.90063
[018] Train 2132736.54648 | Val MSE 1949348.22906
[019] Train 2124243.01719 | Val MSE 1944665.02156
[020] Train 2127486.83818 | Val MSE 1939844.12156
학습 완료 gat.pt 에 저장
```

해야할 것

- MSE가 과도하게 크게 나오는 현상 해결
- 하이퍼 파라미터 튜닝
- MARIO 모델 분석